

Experiment 8

Student Name: Manasvi Sharma
Branch: BE CSE
Semester: 6th
Subject Name: Full Stack Development

UID: 23BCS11815
Section/Group: KRG 3A
Date of Performance: 06/04/26
Subject Code: 23CSH-309

Aim:

To understand and implement security in a full-stack application (LivePoll System) using Spring Security, OAuth (Google Login), role-based access control (RBAC), and integration with a React frontend.

Objective:

1. To understand the importance of security in backend applications
2. To learn how Spring Security manages authentication and authorization
3. To differentiate between authentication and authorization
4. To secure REST APIs using custom security configurations
5. To implement OAuth 2.0 (Google Login)
6. To apply role-based access control (USER, ADMIN)

Implementation/Code:

AdminPollController.java

```
package com.ecotrack.livepoll.controller;

import com.ecotrack.livepoll.auth.AppUserPrincipal;
import com.ecotrack.livepoll.dto.PollDtos;
import com.ecotrack.livepoll.service.PollService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
```

```
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
@RequestMapping("/api/admin/polls")
```

```
@PreAuthorize("hasRole('ADMIN')")
```

```
public class AdminPollController {
```

```
    private final PollService pollService;
```

```
    public AdminPollController(PollService pollService) {
```

```
        this.pollService = pollService;
```

```
    }
```

```
    @PostMapping
```

```
    public ResponseEntity<PollDtos.PollResponse> createPoll(@Valid @RequestBody
    PollDtos.CreatePollRequest request,
```

```
                    @AuthenticationPrincipal AppUserPrincipal
```

```
principal) {
```

```
        return ResponseEntity.status(HttpStatus.CREATED)
```

```
            .body(pollService.createPoll(request, principal.getUsername()));
```

```
    }
```

```
    @PostMapping("/{id}/close")
```

```
    public ResponseEntity<PollDtos.PollResponse> closePoll(@PathVariable Long id) {
```

```
        return ResponseEntity.ok(pollService.closePoll(id));
```

```
    }
```

```
    @DeleteMapping("/{id}")
```

```
    public ResponseEntity<Void> deletePoll(@PathVariable Long id) {
```

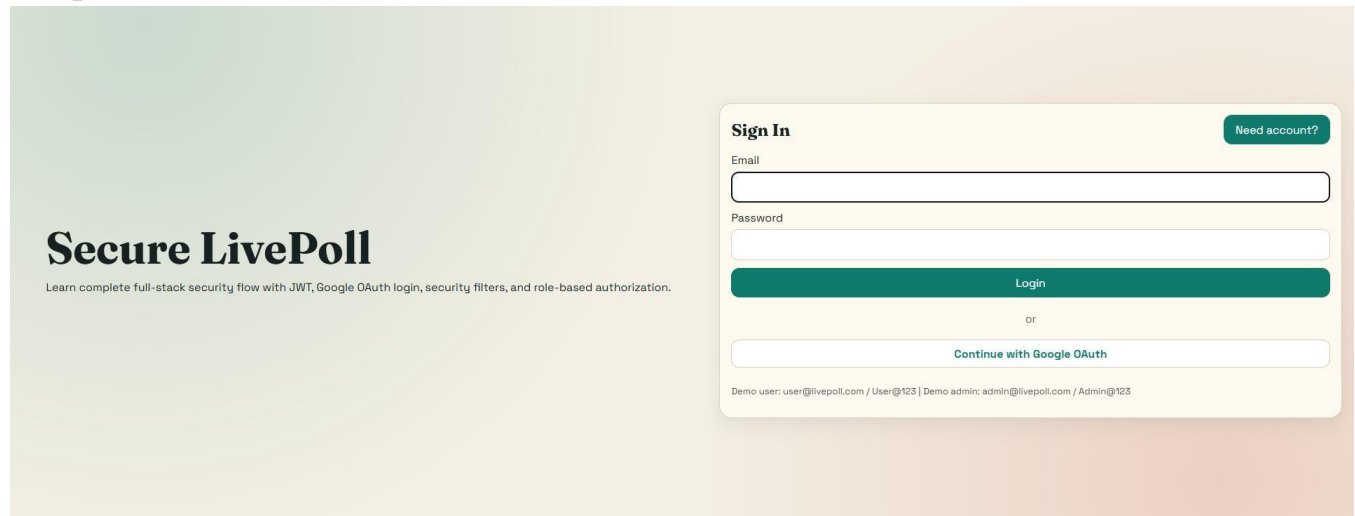
```
        pollService.deletePoll(id);
```

```
        return ResponseEntity.noContent().build();
```

```
    }
```

```
}
```

Output:



Learning Outcome:

- Learned how Spring Security secures backend APIs
- Understood authentication vs authorization clearly
- Implemented OAuth 2.0 login using Google
- Gained experience with role-based access control (RBAC)
- Learned how to integrate React frontend with secured backend
- Understood complete request flow with security filters
- Improved debugging and analysis of secured applications